Report on Career Pathways and the Future of Low-Code Development

Roshan S

241501170

AIML

Abstract

This report explores the career pathways created by low-code technologies, the technical and professional skills they require, and the likely future of low-code development. It describes the main roles, learning and certification routes, and the changes expected as AI and governance become part of every platform. It also discusses why low-code is a useful skill for a student of artificial intelligence and machine learning, using the Gym Membership application built in this lab as a portfolio example.

1. Introduction

As low-code platforms become mainstream, a new set of roles has appeared around building, governing and extending them. Low-code does not remove the need for technical skill. It moves the emphasis from writing syntax to problem solving, data modelling, process design and integration, and it makes those skills valuable to a wider range of people.

2. Career Pathways

Role	Responsibility	Key Skills
Low-Code Developer	Builds applications with forms, reports, workflows and scripting	Data modelling, Deluge or similar scripting, UI design, testing
Citizen Developer	Builds simple tools to solve departmental problems	Domain knowledge, basic logic, platform fundamentals

Role	Responsibility	Key Skills
Business Analyst	Maps business processes into automated workflows	Requirement gathering, process modelling, communication
Integration Developer	Connects low-code apps to other systems	REST APIs, JSON, OAuth, middleware
Solution Architect	Designs solutions combining low-code and pro-code	System design, security, scalability, governance
Platform Administrator	Manages users, roles, environments and compliance	RBAC, audit, deployment management
Pro-Code Extension Developer	Builds custom functions and services for the platform	Java, Python or Node.js, serverless platforms such as Zoho Catalyst
AI and Automation Specialist	Adds AI, analytics and hyperautomation	Machine learning basics, analytics tools, prompt design

The Gym Membership application touches several of these roles at once: the form and nine reports are low-code development, the permission sets are platform administration, and the Catalyst fee calculator is pro-code extension.

3. Skills to Build

Technical Skills	Professional Skills
Relational data modelling	Requirement analysis and problem solving
Workflow scripting (for example Deluge)	Communication with stakeholders
REST APIs and integration	Process thinking and documentation
Security and role-based access	Project and time management
Cloud and DevOps basics	Continuous learning
AI and analytics fundamentals	

Technical depth gives credibility when applications become complex, while professional skills decide how well an application fits the real needs of its users.

4. Certification and Learning Routes

- Vendor certifications and learning paths, such as those offered for Zoho Creator.
- Courses on other platforms such as Microsoft Power Platform, OutSystems, Mendix and Appian, since core skills transfer between platforms.
- A portfolio of working applications, such as the Gym Membership Enrollment application and Reports Suite of this lab, published with documentation on GitHub.
- Hackathons, internships and community forums.

5. Future of Low-Code Development

- **AI-native building:** describing an application in plain language and refining it by conversation will become common.
- **Convergence of low-code and pro-code:** professional developers will use low-code for speed and write code only where it is needed.
- **Stronger governance:** enterprises will manage low-code as a controlled part of their IT landscape.
- **Wider adoption:** vertical templates will serve more industries such as fitness, hospitality, healthcare and education.
- **Role evolution:** routine form and report building will be increasingly automated, so value moves to architecture, integration, data and AI skills.

6. Relevance to an AI and ML Student

For a student of artificial intelligence and machine learning, low-code is a fast way to turn models into usable products. A trained model can be exposed through an API and consumed by a low-code front end that already provides forms, user management and dashboards. For example, a churn prediction model trained on gym membership data could be called from the Gym Membership application and its score shown on the manager's dashboard. Combining machine learning skills with low-code delivery and API integration is a strong and practical profile.

7. Conclusion

Low-code development opens a broad set of careers that reward a mix of technical depth and business understanding. As AI makes building easier, long-term value will lie in designing sound data models, integrations, security and intelligent automation. Students who learn the fundamentals and build a visible portfolio will be well placed.

8. Summary

The report lists eight career roles around low-code, the technical and professional skills they need, and routes for learning and certification. It expects the future to bring AI-native building, closer ties between low-code and pro-code, stronger governance and wider adoption, with value shifting to architecture, integration, data and AI.

9. Learning Outcome

This report helps the learner identify career options in low-code development, plan the skills and portfolio needed for them, and understand how the field is likely to change.